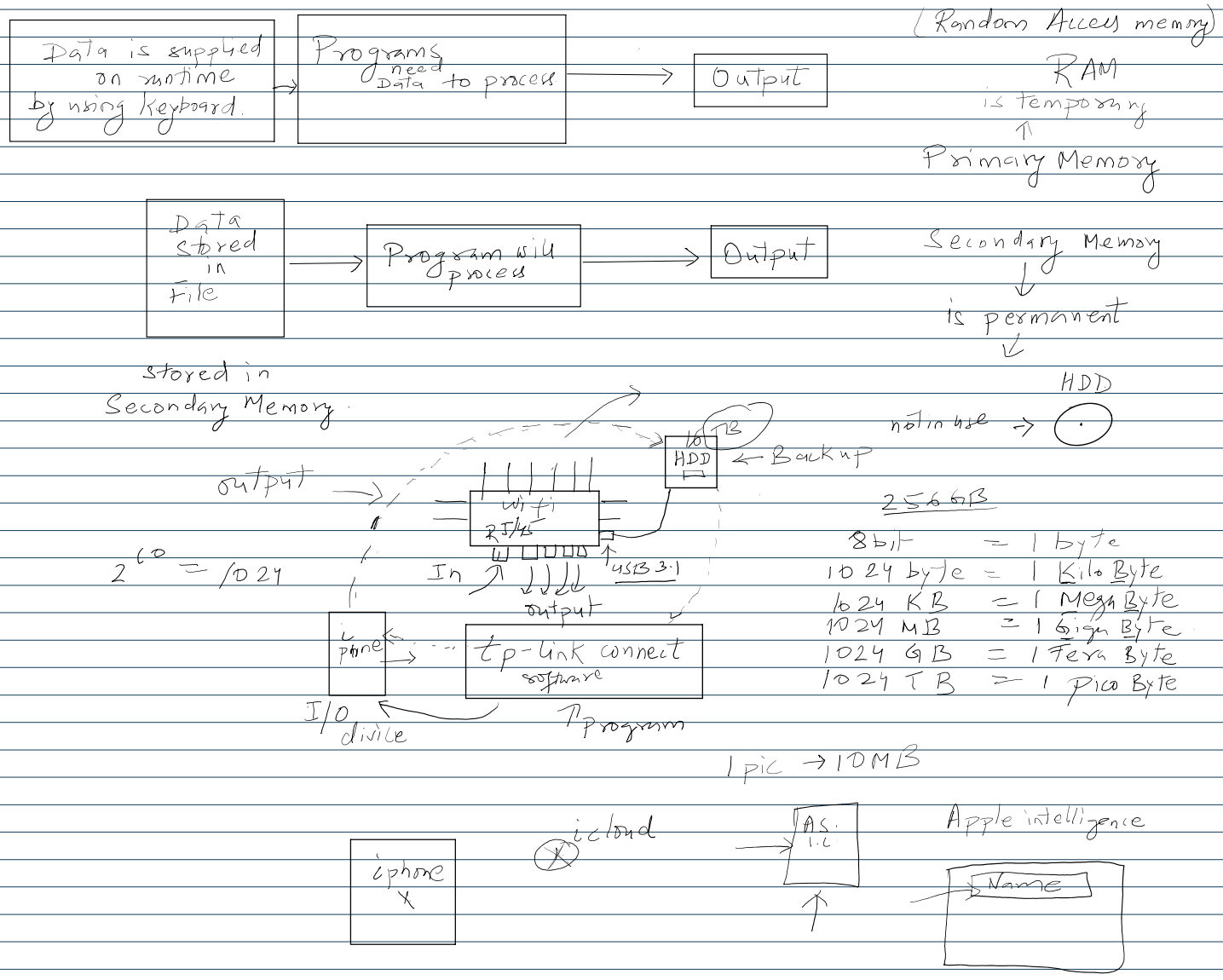
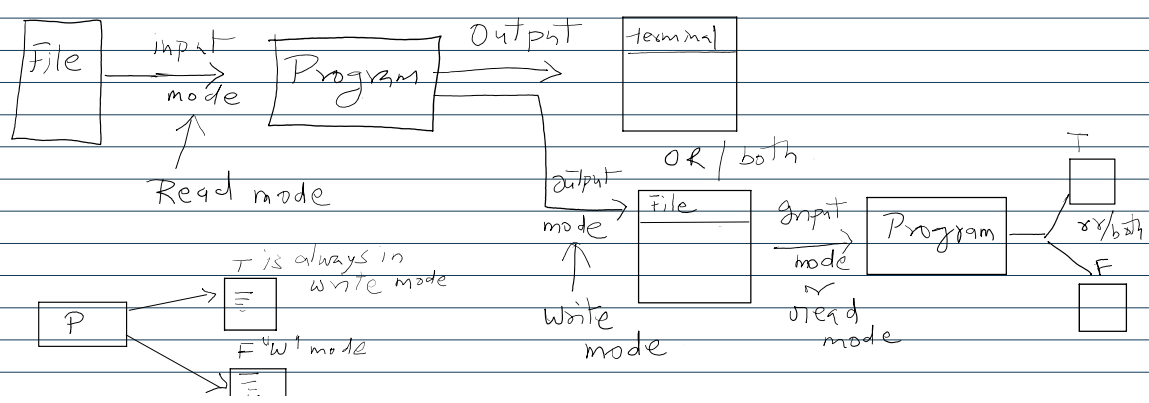
Concept of File Handling using Python

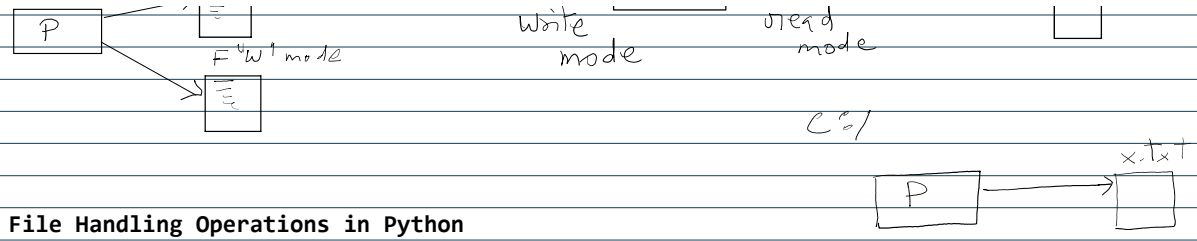
06 May 2025 20:10

File handling in python is the process of working with files for reading, writing, updating and deleting data stored on disk. Python provides built in functions and methods to handle file easily using the open() function.



⇒ Frontend ⇒ Backend

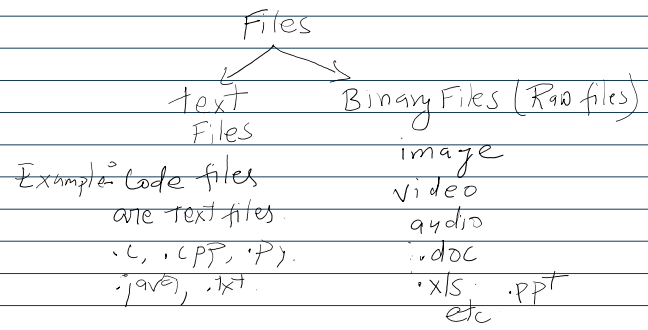




Basic File Handling Operations in Python

```
file = open("filename with address", "mode")
```

- "r" => Read mode(default mode)
- "w" => Write mode(creates a new file or truncates existing, it means removes the existing data, write the new data).
- "a" => Append(add content to the end)
- "x" => Exclusive mode, create a new file; fails if file exists.
- "b" => Binary mode(e.g., "rb" or "wb")
- "t" => text mode(default).
- "+" => Read and write(Combine with others(e.g., 'r+', 'w+') to allow both reading and writing.



Opening and Closing a text File

To open a file in python, we can use open () function The syntax of open function is as follows:

```
File_object = open(file_name, access_mode)
```

This function returns a file object called file handle, which is stored in the variable File_object. We can use this variable to transfer data to and from the file (read and write) by calling the functions defined in the Python's 'io' module. If the file does not exist, the above statement creates a new empty file and assigns it the name we specify in the statement.

The File_object has certain attributes that tells us basic information about the file such as:

<File_object.closed>: Returns True if the file is closed and False otherwise.
<File_obejct.mode>: Returns the access mode in which the file was opened.
<File_object.name>: Returns the name of the file.

The file_name should be the name of the file that has to be opened. If the file is not in the current working directory, then we need to specify the complete path of the file with its name.

The access_mode Is an optional argument that represents the mode in which the file has to be accessed by the program. It is also referred to as processing mode. here mode means the operation for which the file has to be opened like <r> for reading and <w> for writing, <+> for both reading and writing, <a> for appending at the end of the existing file. The default is the read mode. In addition, we can specify whether the file will be handled as Binary<b> or text mode. By default, files are opened in text mode. That means strings can be read or written. Files containing non textual data are

opened in Binary mode. That means read or write are performed in terms of bytes.

```
myObject = open("./FileHandling/Files/myfile.txt", "+w")
if not myObject.closed:
    myObject.close()
else:
    print('File is closed')
```

Another method to open a file:

```
# File is closed automatically
with open("./FileHandling/Files/myfile2.txt", "w") as fobj:
    fobj.write("Hello! I am here!!!")
```

The advantage of using with clause is that any file that is opened using this clause is closed automatically once the control comes outside the with clause.

```
# File is closed automatically
with open("./FileHandling/Files/myfile2.txt", "w") as fobj:
    fobj.write("Hello! I am here!!!")

with open("./FileHandling/Code/fh1.py", "r") as fobj:
    content = fobj.read()
    print(content)
```

← for reading information in the file.

Here we don't have to close the file explicitly using close () statement. Python will automatically close the file.

Writing to a Text File

- "write()" - for writing single string.
- "writeline()" = for writing a sequence of strings.

"write()" Method takes a string as an argument and writes it to the text file. It returns the number of characters being written on single execution of write() method. Also, we need to add a newline character('\n') at the end of every sentence to mark the end of the line.

```
fileName = input('Enter file name: ')
with open("./FileHandling/Files/"+fileName, "+a") as fileObject:
    while True:
        line = input('Type a line or press "x" or "X" to exit: ')
        if line.lower() == 'x':
            exit()
        fileObject.write(line+"\n")
```

```
FileHandling > Files > poem.txt
NOTHING GOLD CAN STAY
BY ROBERT FROST
Nature's first green is gold,
Her hardest hue to hold.
Her early leaf's a flower;
But only so an hour.
Then leaf subsides to leaf.
So Eden sank to grief.
So dawn goes down to day.
Nothing gold can stay.
"Hope" is the thing with feathers
That preaches in the soul.
And sings the tune without the words,
And never stops at all.
```

```
fileName = input('Enter file name: ')
with open("./FileHandling/Files/"+fileName, "+a") as fileObject:
    while True:
        line = input('Type a line or press "x" or "X" to exit: ')
        if line.lower() == 'x':
            exit()
        print("Number of characters written in file:", fileObject.write(line+"\n"))
```

Output:

```
Enter file name: poem.txt
Type a line or press "x" or "X" to exit: And sweetest in the gale is heard;
Number of characters written in file: 35
Type a line or press "x" or "X" to exit: And sore must be the storm.
Number of characters written in file: 28
Type a line or press "x" or "X" to exit: x
```

The writelines() method: This method is used to write multiple strings to a file. We need to pass an iterable object, like list, tuple etc. containing strings to the writelines() method. The writelines() does not return the number of characters written in the file.

```
fileName = input('Enter file name: ')
lineList = []
with open("./FileHandling/Files/"+fileName, "+a") as fileObject:
    while True:
        line = input('Type a line or press "x" or "X" to exit: ')
        if line.lower() == 'x':
            break
        lineList.append(line+"\n")
    fileObject.writelines(lineList)
```

Output:

```
FileHandling > Files > poem1.txt
I've heard it in the chilliest land
And on the strangest sea;
Yet, never, in extremity,
It asked a crumb of me.
```

Reading from a Text File

We can write a program to read the contents of a file. before reading a file, we must make sure that the file is opened in "r", "r+", "w+" or "a+" mode. There are three ways to read the contents of the file.

The read() method

This method is used to read a specific number of bytes of data from the data file. The

syntax of read() Method is:

```
File_object.read(n)
```

If no argument or a negative number is specified in read (is closed parenthesis The entire file content is read. The syntax is as follows:

```
File_object.read()
```

```
fileName = input('Enter File Name: ')
with open("./FileHandling/Files/"+fileName, "r") as fileObject:
    while True:
        line10 = fileObject.read(10)
        if line10 != '':
            print(line10)
        else:
            break
```

Output:

```
Enter File Name: poem1.txt
I've heard
 it in the
 chilliest
 land
And
on the str
angest sea
;
Yet, nev
er, in ext
remity,
It
 asked a c
rumb of me
.
```

```
fileName = input('Enter File Name: ')
with open("./FileHandling/Files/"+fileName, "r") as fileObject:
    fullFile = fileObject.read()
    vowels = 'AEIOUaeiou'
    count = 0
    for c in fullFile:
        if c in vowels:
            count+=1
    print(fullFile)
    print('\nNumber of vowels:', count)
```

Output:

```
Enter File Name: poem1.txt
I've heard it in the chilliest land
And on the strangest sea;
Yet, never, in extremity,
It asked a crumb of me.


Number of vowels: 32
```

Home work:

WAP in Python to take the name file from the user, create the file in the Folder, and write a story in the file.

WAP in Python to read the file, and print all the words whose length is greater than 5 and less than 10, calculate the number of vowels and consonants of each word. And find the greatest word from the file.

Deadline: Wednesday 8PM.

Creating and traversing a text File.

Creating a file, Writing data, and reading data. We are able to move the file object by using seek(N) method, by passing the byte number and tell() updates us about, our

file object is at which position.

```
fileName = input('Enter file name: ')
fileObject = open('./FileHandling/Files/' + fileName, "r+")
# print()
# while True:
#     line = input('Enter any sentence: ')
#     fileObject.write(line)
#     fileObject.write('\n')
#     choice = input('Do you wish to enter more data? (y/n): ')
#     if choice in ('n','N'):
#         break

print("The byte position of the file object is", fileObject.tell())
fileObject.seek(10) #places file object at beginning of the file
print("The byte position of the file object after seek() is", fileObject.tell())
print()
print("READING DATA FROM THE FILE")
content = fileObject.read()
print(content)
print("The byte position of the file object is", fileObject.tell())
fileObject.seek(30) #places file object at beginning of the file
print("The byte position of the file object after seek() is", fileObject.tell())
content = fileObject.read()
print(content)
fileObject.close()
```

HOME WORK
Q7.2 TO 7.6 PG-444 AND 445

Binary File Mode

To handle binary file mode, we need to study The Pickle Module of Python

We know that python considers everything as an object, so all data type, including list, tuple, dictionary, etc., are also considered as objects. During execution of a program, we may require to store current state of variables, so that we can retrieve them later to its present state. Suppose you are playing a video game and after some time you want to close it. So the program should be able to store the current state of the game, including current level or stage, your score etc. as a Python object.
Likewise, you may like to store a python dictionary as an object to be able to retrieve later. to save any object structure along with data python provides a module called pickle. The module pickle is used for serializing and deserializing any python object structure.
Pickling is a method of preserving food items by placing them in some solution which increases the shelf life. In other words, it is a method to store food items for later consumption.
Serialization is the process of transferring data or an object in memory (RAM) to a stream of bytes called byte stream. These byte streams in a binary file can then be stored in a disk or in a database, or sent through a network. Serialization process is also called pickling.
De-serialisation or unpickling is the inverse of pickling process where a white stream is converted back to Python object.
The pickle module deals with binary files. Here, data are not written, but dumped. And similarly, data are not read, but loaded. The pickle module must be imported to load and dumped data. The pickle module provides two method, dump() and load() to work with binary files for pickling and unpickling, respectively.

The dump() method
This method is used to convert (pickling) Python objects for writing data in a binary file. The file in which data are to be dumped need to be opened in a binary right mode (WB).
Syntax for dump () is as follows:

dump(data_object, file_object)
Where data_object is the object that has to be dumped to the file with the file handle name file_object.

IDE (Integrated Development Environment)

→ Runtime Environment
Editor to edit the code
Compiler

Examples:- VSCode, Pycharm

Microsoft

Pycharm

Jetbrains

For example:

```
import pickle as P
student = {'rollNum':1, 'name':'Jiya', 'section':'E', 'class':8}
fileName = input('Enter file name: ')
with open('./FileHandling/Files/'+fileName,'wb') as fo:
    P.dump(student, fo)
```

Output:

Enter file name: students.dat
file:

```
FileHandling > Files > students.dat
1  ⁄EOT⁄7NULNULNULNULNULNUL}⁄(⁄BELrollNum⁄KSOH⁄EOTname⁄⁄EOTJiya⁄⁄BELsection⁄⁄SOHE⁄⁄ENQclass⁄KBSu.
```

↑ content of binary file displayed by VSCode.

The load() method
This method is used to load (unpickling) data from a binary file. The file to be loaded is opened in binary read mode (Rb) mode.
Syntax of load () is as follows:

```
Store_object = load(file_object);
```

Here the pickled Python object is loaded from the file having a file handled name file _object and is stored in a new file handle called store _object. For example:

```
import pickle as P
print('the data stored in binary file: ')
fileName = input('Enter file name to read: ')
with open('./FileHandling/Files/'+fileName, 'rb') as fb:
    data = P.load(fb)
    print(data)
```

Output:

```
Enter file name to read: students.dat
{'rollNum': 1, 'name': 'Jiya', 'section': 'E', 'class': 8}
```

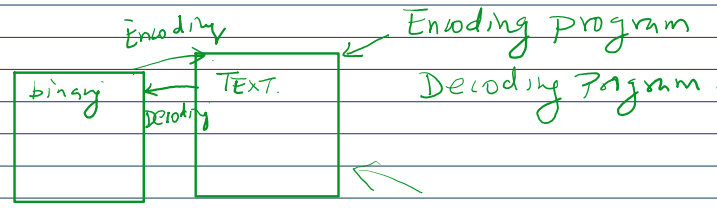
Q2 → C:\> python P72.py poem1.txt poemout.txt ↵

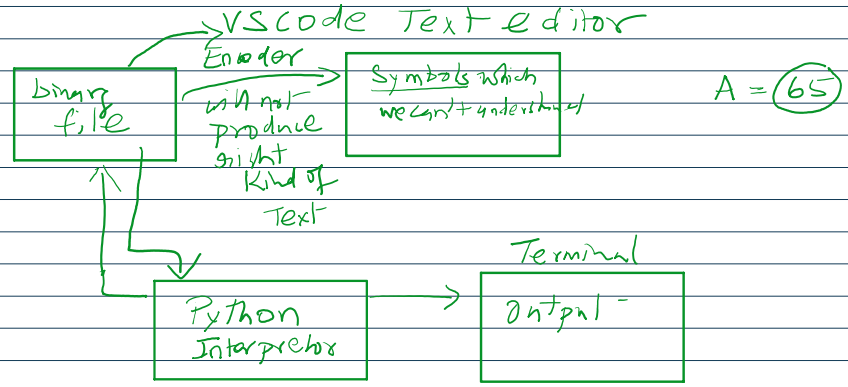
Q3 → C:\> python P72.py ↵
Enter input fileName: _____
Enter output fileName: _____

Program to accept a record of an employee from the user and appends it in the binary file. Thereafter, the records are read from the binary file and displayed on the screen using the same object. The user may enter as many records as they wish to. The program also displays the size of binary files before starting with the reading process.

Text Files uses Encoding and decoding techniques to process the data.

Encoding → ← Encoding program






```
# Program to accept a record of an employee from the user and appends it in the
binary file.
# Thereafter, the records are read from the binary file and displayed on the screen
using the
# same object. The user may enter as many records as they wish to. The program also
displays
# the size of binary files before starting with the reading process.
import pickle as P
print("working with binary files")
fileName = input('Enter file name: ')
recordNum = 1
print('Enter records of employees')
print()
with open('./FileHandling/Files/'+fileName,'ab' ) as fileObject:
    while True:
        print('Record number:', recordNum)
        empno = input('Enter employee number: ')
        ename = input('Enter employee name: ')
        ebasicSal = int(input('Enter basic salary: '))
        allowance = int(input('Enter allowances: '))
        totalSal = ebasicSal + allowance
        print('Total salary: ', totalSal)
        edata = [empno, ename, ebasicSal, allowance, totalSal]
        P.dump(edata, fileObject)
        recordNum += 1
        ans = input('Do you wish to enter more records (y/n)? ')
        if ans.lower() == 'n':
            print('Record entry is OVER')
            print()
            break
    # Retrieving the size of the file.
    print('Size of the binary file (in bytes): ', fileObject.tell())
# Reading the employees record from the file using load() module.
print('Now reading the employee records from the file.')
print()
readrec = 1
try:
    with open('./FileHandling/Files/'+fileName, 'rb') as fileObject:
        while True:
            edata = P.load(fileObject)
```

```
            print("Record Number: ", readrec)
            print(edata)
            readrec += 1
except EOFError:
    print()
    print('End of file reached')
    print()
```

Output:

```
working with binary files
Enter file name: employee.bin
Enter records of employees

Record number: 1
Enter employee number: E0008
Enter employee name: Yash
Enter basic salary: 54000
Enter allowances: 13000
Total salary:  67000
Do you wish to enter more records (y/n)? n
Record entry is OVER

Size of the binary file (in bytes):  335
Now reading the employee records from the file.

Record Number:  1
['E0001', 'Harsh', 50000, 10000, 60000]
Record Number:  2
['E0002', 'Vaanya', 40000, 7600, 47600]
Record Number:  3
['E0003', 'Jiya', 60000, 12000, 72000]
Record Number:  4
['E0004', 'Keshav', 54000, 24000, 78000]
Record Number:  5
['E0005', 'Aadya', 70000, 5000, 75000]
Record Number:  6
['E0006', 'Jack', 40000, 3000, 43000]
Record Number:  7
['E0007', 'Raj', 5000, 12000, 17000]
Record Number:  8
['E0008', 'Yash', 54000, 13000, 67000]

End of file reached
```

Improved version of above code:

```
# Program to accept a record of an employee from the user and appends it in the
binary file.
# Thereafter, the records are read from the binary file and displayed on the screen
using the
# same object. The user may enter as many records as they wish to. The program also
displays
# the size of binary files before starting with the reading process.
import pickle as P
print("working with binary files")
fileName = input('Enter file name: ')
recordNum = 1
print('Press 1 to write records in the file')
print('Press 2 ro read all records from the file')
choice = int (input('Enter your choice (1, 2)? '))
match choice:
    case 1:
        print('Enter records of employees')
```

```
        print()
        with open('./FileHandling/Files/'+fileName,'ab' ) as fileObject:
            while True:
                print('Record number:', recordNum)
                empno = input('Enter employee number: ')
                ename = input('Enter employee name: ')
                ebasicSal = int(input('Enter basic salary: '))
                allowance = int(input('Enter allowances: '))
                totalSal = ebasicSal + allowance
                print('Total salary: ', totalSal)
                edata = [empno, ename, ebasicSal, allowance, totalSal]
                P.dump(edata, fileObject)
                recordNum += 1
                ans = input('Do you wish to enter more records (y/n)? ')
                if ans.lower() == 'n':
                    print('Record entry is OVER')
                    print()
                    break
            # Retrieving the size of the file.
            print('Size of the binary file (in bytes): ', fileObject.tell())
    case 2:
        # Reading the employees record from the file using load() module.
        print('Now reading the employee records from the file.')
        print()
        readrec = 1
        try:
            with open('./FileHandling/Files/'+fileName, 'rb') as fileObject:
                while True:
                    edata = P.load(fileObject)
                    print("Record Number: ", readrec)
                    print(edata)
                    readrec += 1
        except EOFError:
            print()
            print('End of file reached')
            print()
    case _:
        print('Wrong Option selected')
```

Output:

```
working with binary files
Enter file name: employee.bin
Press 1 to write records in the file
Press 2 to read all records from the file
Enter your choice (1, 2)? 2
Now reading the employee records from the file.

Record Number:  1
['E0001', 'Harsh', 50000, 10000, 60000]
Record Number:  2
['E0002', 'Vaanya', 40000, 7600, 47600]
Record Number:  3
['E0003', 'Jiya', 60000, 12000, 72000]
Record Number:  4
['E0004', 'Keshav', 54000, 24000, 78000]
Record Number:  5
['E0005', 'Aadya', 70000, 5000, 75000]
Record Number:  6
['E0006', 'Jack', 40000, 3000, 43000]
Record Number:  7
['E0007', 'Raj', 5000, 12000, 17000]
Record Number:  8
['E0008', 'Yash', 54000, 13000, 67000]

End of file reached
```

Home work:
Exercise

1. Differentiate between:
a) text file and binary file
b) readline() and readlines()
c) write() and writelines()

2. Write the use and syntax for the following methods:
a) open()
b) read()
c) seek()
d) dump()

3. Write the file mode that will be used for opening the following files. Also, write the Python statements to open the following files:
a) a text file “example.txt” in both read and write mode
b) a binary file “bfile.dat” in write mode
c) a text file “try.txt” in append and read mode
d) a binary file “btry.dat” in read only mode.

4. Why is it advised to close a file after we are done with the read and write operations? What will happen if we do not close it? Will some error message be flashed?

5. What is the difference between the following set of statements (a) and (b):
a) P = open(“practice.txt”,”r”)
P.read(10)
b) with open(“practice.txt”, “r”) as P:
x = P.read()

6. Write a command(s) to write the following lines to the text file named hello.txt. Assume that the file is opened in append mode.
“ Welcome my class”
“It is a fun place”
“You will learn and play”

7. Write a Python program to open the file hello.txt used in question no 6 in read mode to display its contents. What will be the difference if the file was opened in write mode instead of append mode?

8. Write a program to accept string/sentences from the user till the user enters “END” to. Save the data in a text file and then display only those sentences which begin with an uppercase alphabet.

9. Define pickling in Python. Explain serialization and deserialization of Python object.

10. Write a program to enter the following records in a binary file:
Item_No integer
Item_Name string
Qty integer
Price float
Number of records to be entered should be accepted from the user. Read the file to display the records in the following format:
Item No:
Item Name :
Quantity:
Price per item:

Amount: [To be calculated as Price * Qty.]